

DAY 16 — ASYNC + PARALLEL EXECUTION (FINAL VERSION)

GOAL

Upgrade from:

Step 1 → wait → Step 2 → wait → Step 3

to:

All steps run in parallel

(Faster system)

WHY THIS IS IMPORTANT

Example:

Query: max price and explain trend

Before:

Excel → wait

RAG → wait

→ slow

After:

Excel + RAG run together 

→ faster response

STEP 1 — MODIFY AGENT (CORE CHANGE)

 app/agent.py

ADD IMPORT

import asyncio

ADD ASYNC TOOL EXECUTOR

async def execute_tool(self, tool, task):

```

if tool == "EXCEL" and self.excel:

    return await asyncio.to_thread( self.excel.smart_query, task, generate)

elif tool == "PDF_TABLE" and self.pdf_table:

    return await asyncio.to_thread( self.pdf_table.smart_query, task, generate )

elif tool == "RAG":

    return await asyncio.to_thread( self.rag.search, task )

return "Unknown tool"

```

✅ UPDATE run() TO ASYNC VERSION

👉 Replace FULL run() function:

```

async def run(self, query):

    # Step 1: memory

    history = self.memory.get()

    past_context = ""

    for item in history:

        past_context += f"User: {item['user']}\nAssistant: {item['response']}\n"

    enriched_query = f""

    Conversation:

    {past_context}

    Current:

    {query}""

    # Step 2: planning

    plan = self.planner.create_plan(enriched_query)

    if not plan:

        plan = [("EXCEL", query)]

    print("FINAL PLAN:", plan)

    # ✅ Step 3: RUN ALL TASKS IN PARALLEL

    tasks = []

```

```

for tool, task in plan:

    tasks.append(self.execute_tool(tool, task))

results_raw = await asyncio.gather(*tasks)

results = []

for i, res in enumerate(results_raw):

    print(f"STEP {i+1} RESULT:", res)

    results.append(f"Step {i+1}:\n{res}")

# Step 4: final answer

final_prompt = f"""
Context:
{results}

Question:
{query}"""

final_answer = generate(final_prompt)

# Step 5: evaluation

evaluation = self.evaluator.evaluate( query=query, context=str(results), answer=final_answer )

print("\nEVALUATION:\n", evaluation)

# Step 6: logging

self.logger.log({

    "query": query,

    "steps": results,

    "answer": final_answer,

    "evaluation": evaluation })

# Step 7: memory

self.memory.add(query, final_answer)

return final_answer

```

STEP 2 — UPDATE API (IMPORTANT)

 app/api.py

👉 Change endpoint:

```
@app.post("/ask")
```

```
async def ask(q: Query):
```

```
    answer = await agent.run(q.query)
```

```
    return {
```

```
        "query": q.query,
```

```
        "answer": answer    }
```

✅ STEP 3 — TEST

Run:

```
uvicorn app.api:app --reload
```

✅ Test queries:

```
>> max price and explain trend
```

```
>> average sales and summarize document
```

```
>> compare excel and pdf trends
```

✅ EXPECTED IMPROVEMENT

Before:

Slow execution

Sequential

After:

Parallel execution ✅

Faster response ✅

Better scalability ✅

✅ FINAL FLOW NOW

User Query

↓

Planner

↓

Parallel execution (NEW)

└─ Excel

└─ RAG

└─ PDF_TABLE

↓

Results combined

↓

LLM Answer

↓

Evaluation + Logging + Memory

✅ WHAT YOU BUILT TODAY

Concurrent / Parallel AI agent system

✅ SYSTEM LEVEL NOW

You now have:

✅ AI system

✅ API

✅ Docker

✅ Parallel execution (NEW)

✅ INDUSTRY TERM

Async Agent Execution / Concurrent Tools

Used in:

- LangGraph

- Advanced Copilot systems
 - Production AI pipelines
-

✅ **NEXT**

DAY 17 → Caching (reduce cost + speed boost)

If ready: 👉 say **Day 17**